A Comparison of Circuit Level Optimizations on Shor's Algorithm for Cryptographically Relevant Integers

Abstract

This paper aimed to find the cost estimate of factoring RSA integers greater than 2048 bits using circuit-level optimizations by looking at spacetime volume (in megaqubitdays) and counts of gates which dominate the cost (e.g. toffolis). We found the best plausible combinations of tunable parameters based on optimizations such as windowed arithmetic and runways on 3D gauge surface code by using a simulation. We found that RSA 2048 could be factored when using $12,324$ qubits and 93 days, and $Y = 1.856\text{e}{-11}(X)^{3.258}$ can be used to predict the spacetime volume. In comparison to previous works, our estimations provided a substantial decrease in spacetime volume because of the decrease in the days required to factor. Fedorkia's adder allowed us to greatly decrease the Toffoli count alongside oblivious carry runways, which decreased circuit depth and in turn time. Our work presents a temporary solution of having internet providers switch to larger integers instead of a switch to post-quantum encryption.

1 Introduction

Some of the first mathematicians to consider integer factorization were the Greeks, particularly Euclid, who proved the fundamental theorem of arithmetic, which states that each positive integer can be represented as the product of one or more primes [26]. Several hundred years later, tables containing the factors of various numbers were created. Still, due to their inefficiency, attempts were made to develop structured processes called algorithms to factor any integer systematically [8]. Methods attempting to factor integers made little progress until the problem became evident with the creation of RSA, an encryption algorithm.

The security of RSA, an algorithm used to encrypt messages, relies on keys to encode and decode text. In particular, the public key, visible to all, is made of two positive integers, one of which contains the factors to decrypt the same message [39, 8]. Henceforth, if anybody can factor the positive integer N (of typically large size), they can read all messages encrypted using RSA.

After the emergence of the unbreakable RSA algorithm, the factoring problem was no longer studied as a pure mathematical problem, further increasing its research. One of the more notable algorithms created was the number field sieve, which could factor numbers in a special form in a shorter time compared to previous algorithms such as the elliptic curve and quadratic sieve [36]. Years later, the algorithm would eventually be generalized to factor all integers and called the General Number Field Sieve (GNFS) [36, 8]. Consequently, GNFS remains the fastest classical integer factorization method today for large composite numbers [9, 10].

The field's most significant research advancement was Peter Shor's groundbreaking paper [41], which revealed that integers could be factored, using quantum computers, in polynomial time (compared to the exponential time of all the previous algorithms), revolutionizing how factoring works today. Fast forward to today, and what once seemed impossible—breaking RSA—is now within reach. While quantum computers face challenges such as error and scalability, researchers are actively creating optimizations of Shor's algorithm. Determining what resources quantum computers need to factor relevantly sized integers is essential for estimating when RSA will be breakable, which is essential in an age where all valuable information is kept online.

2 Literature Review

2.1 Quantum Computing

Quantum computers are best defined as a device that manipulates the properties of quantum mechanics. Although quantum mechanics uses wave functions and mathematical operators to predict the most likely properties of particles, quantum computing abstracts much of the physics to make it more manageable to compute.

2.1.1 Qubit and Quantum Circuits

A pure quantum state, also called a qubit, can be seen as the following:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where alpha and beta are complex numbers. To modify the state of a qubit, a gate can be applied, such as the Hadamard gate (see Appendix C for a list of the most common gates). Each gate is represented by a matrix, and multiple gates and qubits make up

quantum circuits. For the purpose of this paper, it is easier to visualize a qubit as being the fundamental unit of information, similar to bits for classical computers.

Shor's algorithm doesn't directly attempt to factor N on a quantum computer but instead tries to find the order/period of a randomly chosen number x where the order of x, r is the least positive integer such that x^r is identical to $1(\bmod N)$[1]. By finding the order of x we can compute the factors of n by using the greatest common denominator (gcd) of x and N ($\gcd(x, N)$)[41]. There are three main components of Shor's algorithm: the Hadamard transform, modular exponentiation, and the Inverse Quantum Fourier Transform (IQFT), often denoted $QFT^{\dagger}$[2] shown in Figure 1 (See Appendix A for the entire algorithm outline).

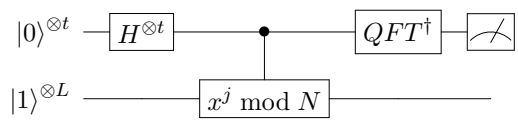


Figure 1: The circuit contains two quantum registers one with t and the other L qubits (denoted by the $\otimes t$ and $\otimes L$. Hadamard gates are applied to the first register shown with the Hadamard transform. Controlled modular exponentiation is applied to the second register, and the inverse QFT is applied to the first register. The last gate represents a measurement of all the qubits in the first register after IQFT.

Shor uses QFT, similar to the discrete Fourier transform, to find the order/period of x. As shown in Figure 1, the IQFT operation, along with many other gates, makes up the entire quantum operation while the classical computer takes turns the order of x into N's factors (during what we call post-processing).

Although Shor's algorithm reduces the time compared to classical computers, estimations require 20 million qubits [22]. Current quantum computers aren't capable of doing these computations due to the number of qubits required and other factors, which we will review in the following section.

2.2 Current Landscape of Quantum Computing

Quantum computers may achieve the basic requirements of a quantum computer through a variety of ways, such as superconducting qubits, ion traps, nuclear magnetic resonance (NMR), and photonic chips. Although qubits are the main factor of quantum computers, other factors such as gate error rate and number of gates must still be considered. Finding a balance between error rates and qubits is necessary when creating quantum computers, as Shor's algorithm and other applications often require both to be extremely high.

Currently, superconducting qubits have seen the most development with IBM's Condor featuring 1,121 qubits [1] and Google's Willow [3, 2], just to name a few, but a fundamental problem with these computers is error.

2.3 Quantum Error Correction

Quantum noise/error is when the operation on a qubit acts differently than its theoretical expectation, and it is due to many factors, such as environmental changes and state preparation issues, just to name a few. These errors can become compounded and lead to

[1] This simply means the remainder of x^r/N is 1.

[2] A function that finds the order (also called the period) given input states.

complete loss of information, which researchers have tried to address using various techniques. One common method is Quantum Error Correction (QEC,) which attempts to find and fix errors caused by gates.

QEC is built on its classical analogous: classical coding theory, which attempts to reduce error primarily through redundancy [40]. Errors on quantum computers can be written as two types of errors[3]: bit-flips and phase-flips, and we use a surface code to fix them. A surface code arranges physical qubits in a 2D/3D structure and uses measurements to detect and fix errors between neighboring qubits [17, 40].

One of the negatives of surface codes is the applicability of non-Clifford gates such as T gates, which need a special process called distillation, which is computationally expensive [20].

2.4 Physical Realization

As such only a few proof of concepts/experiments have been conducted on superconducting qubits [30, 4], NMR [43], ion traps [33], and photonic chips [35] and have all involved externally small numbers (largest number factored was 21) due to the challenges we mentioned. As a result, optimizations have been made to improve its performance, and they can be classified into two categories, theoretical and circuit-level optimizations.

Theoretical optimizations aim to modify the mathematics behind the algorithm in order to reduce theoretical cost compared to circuit-level optimizations, reduce the size of operations on the qubits. Next, we give a brief overview of the history of optimizations made to quantum circuits that apply to the Shor algorithm (the ones used in this paper will be reviewed in depth within the Method section).

A detailed overview of theoretical optimizations can be seen in [37], but we will briefly review the most significant results.

2.5 Review of Optimizations

Shor's Modular Exponentiation function (the second part of Figure 1) is the most computationally expensive part of the circuit (proof provided in Appendix B). Therefore, most of the optimizations to the algorithm are done in this section. Since modular exponentiation can be broken down to modular multiplication, which can be broken into modular additions (proof provided in Appendix B), early research primarily focused on creating a better adder, which would in turn create a more efficient modular exponentiation.

One of the first adders created was the quantum Ripple Carry Adder (RCA) by VBE [44], which works by adding the values in each digit place (e.g., ones, tens, hundreds) and carrying any overflow to the next place, ultimately producing the sum. Using that as a building block, we construct a building block for modular addition which becomes modular multiplication, which then becomes modular exponentiation.

Cuccaro [11] improves on VBE's RCA by cutting a qubit off each operation and canceling out repetitive operations to get an RCA that requires fewer gates and qubits overall, which is why it's most widely used today. Craig Gidney [19] would further optimize Cuccaro's implementation, cutting the number of T gates (more computationally expensive since it is non-Clifford) in half by introducing the AND logical operation (gate) more commonly seen in classical computer science.

While there are other types of adders, such as the Carry Lookahead Adder (CLA,) which often uses rotations implementations of QFT [7, 12], they aren't as efficient as RCAs, yet

[3] Arbitrary errors can be broken down into a linear combination of these two types of errors.

they introduce operations used in other useful adders. The most recent variation of the Cuccaro adder was by Fedoriaka [16], who made use of similar operations as Gidney to reduce the number of qubits required as well as the number of T gates.

	Physical assumptions				Estimated costs		
Historical cost estimate at $n = 2048$	Physical gate error rate	Cycle time (microseconds)	Reaction time (microseconds)	Physical connectivity	Physical qubits (millions)	Expected runtime (days)	Expected volume (megaqubitdays)
Van Meter et al. 2009 [42]	0.2%	49	N/A	planar	6500	410	2600000
Jones et al. 2010 [28]	0.1%	0.25	N/A	planar	620	10	6200
Fowler et al. 2012 [17]	0.1%	1	0.1	planar	1000	1.1	1100
O'Gorman et al. 2017 [34]	0.1%	10	1	arbitrary	230	3.7	850
Gheorghiu et al. 2019 [18]	0.1%	0.2	0.1	planar	170	1	170
Gideny and Ekera 2019 [22]	0.1%	1	10	planar	20	0.31	5.9
Gouzien and Sangouard 2021 [24]	0.1%	1	10	planar	0.013436	177	2.37
(ours) 2025	**0.1%**	**1**	**10**	**planar**	**0.012324**	**93**	**1.15**

Table 1: Comparison of Historical Cost estimates. Cycle time and reaction time refer to assumptions made about the efficiency of QEC. Megaqubitdays is the number of qubits in millions times the number of days estimated. Note: Gouzien and our cost estimates do not rely on a distillation method because distillation is needed with the 3D Surface code used.

Although research has been done on what techniques can be done to make integer factorization plausible on quantum computers, few researchers have estimated the physical cost of factorization given current capabilities and techniques. Table 1 shows the historical estimates of factoring $n = 2048$ bit RSA integers, and most noticeable is the massive reduction with Gouzien and Sangouard's (referred to as GS in the later parts of the paper) estimation compared to previous estimations. By comparison, Gideny and Ekera's (GE) work, the most cited and accepted of the papers listed, estimated 20 million qubits compared to the measly $13,436$ qubits by GS. This is most notable due to his use of 3D surface code, as well as quantum memory, which removed the need for T state distillation[4], yet little is known about the scalability of his estimation as he only factored as high as 2048 bit integers. Furthermore, very few estimations were done surrounding the various modifications of Shor's algorithm for other cryptographically relevant problems, such as the discrete logarithm.

This leads us to our question: How have recent advancements of quantum computers improved the physical cost in terms of spacetime volume (measured in megaqubitdays[5]) and gate count for factoring RSA integers ≥ 2048 bits using Ekera and Hastad's variation of Shor's algorithm?

We attempt to answer this question by calculating the physical cost of almost all plausible combinations of optimization variables for different cryptographic schemes. We hypothesize that recent circuit-level and algorithmic optimizations for Shor's algorithm have significantly reduced the physical cost, measured in megaqubitdays and gate count, of factoring RSA integers ≥ 2048 bits. Specifically, these advancements will lead to large reductions in both qubit count and circuit depth, yielding improvements in physical cost by a notable factor over prior estimates.

3 Method

In order to better understand the progress made with circuit-level optimizations, we will conduct a simulation (experimental design) that accounts for various combinations of parameters at the optimization level and will be compared to previous work (secondary data

[4]Distillation is a way for us to create accurate non-Clifford states by combining several imperfect noisy states.

[5]Megaqubitdays is the number of qubits in millions times the number of days estimated.

analysis). We make the same physical assumptions as previous papers [22, 24] of 1e-3 physical gate error rate, 1 microsecond cycle time between measurements, and 10 microseconds for reaction time. For cost estimation, refer to Section 3.2.

3.1 Our Construction

3.1.1 Base Algorithm

The algorithm we use is based on a modification of Shor's original algorithm [41], which was previously mentioned [14]. While there are other algorithms that have better theoretical performance in certain scenarios [38, 37, 15], we chose this algorithm for its best theoretical performance over most medium-sized integers [13]. Moving forward, we give a brief overview of both these algorithms (for a full outline see Appendix A.1).

Shor's algorithm works by turning the factoring of a composite number into a problem that is easier for quantum computers but very difficult for classical (regular) computers, called order finding.

The order of an element a in Z is $\mathbf{r}$ such that $a^{\mathbf{r}}$ is identical to 1 mod N, which essentially means the remainder of a/N is just 1. Finding the order ($\mathbf{r}$) of a helps us find the factors of N (See Appendix A.1). Although Shor's algorithm and its variations rely on classical post-processing, their cost is not accounted for since we only care about the order-finding circuit conducted on the quantum computer. Again, as mentioned in Section 2.5, the cost of applying Hadamards and IQFT is known to scale linearly, which is why it is not accounted for within this paper.

3.1.2 Short Log Modification

We chose the Ekera and Hastad's (EH) modification of Shor's algorithm over other newer optimizations, such as Regev's algorithm, since its improvements compared to EH are limited, and many optimizations that were usable for Shor's algorithm aren't for Regev's [15].

In [14] Ekera and Hastad (EH) applied the previously mentioned modification of Shor's algorithm to discrete logs (extremely similar to the order finding mentioned in the section above) to the order finding problem and found that it was more efficient to take so called "short discrete logs" instead of directly finding the order. It turns out that the implementation of "short discrete logs" is very similar to Shor's original algorithm, and the theoretical modifications of Shor's algorithm also apply to this modification.

Further optimizations by Ekera [13] made the probability of success much higher, as well as provided the number of exponentiations n_e [6] required for common sizes of RSA integers, which we use to estimate the number of gates required. The formula he provided in his Table 4 was used for our estimations.

3.1.3 Coset Representation

First introduced by Zalka in 2006 [46] this optimization allows for parallelization of Shor's algorithm by essentially adding more qubits to a new register that would contain fewer gates because it would change from modular addition to regular addition. Although, since it is an approximation of the actual circuit/gates required, the accuracy of the circuit depends on the padding or number of qubits we add as we define c_{dev}. We use the optimized

[6] n_e is the size of the exponent for exponentiation. It would be the size of j in Figure 1

version of the coset initialization from [20], which uses a measurement-based approximation formalized in [31].

3.1.4 Windowed Arithmetic

"Windowing" in the classical sense allows us to merge operations by looking up their results from a table of precomputed values. Its quantum version was initially introduced by Craig Gideny in [21], where he also applies it to modular exponentiation, significantly reducing the number of gates and qubits required.

We use windowing for multiplication and exponentiation, where w_e and w_m denote the size of the window and can be optimized for during the simulation. A window for multiplication would turn a group of additions into lookup additions, which instead of doing the addition, looks up the answer and insert it into the register. Although previous research decided to test windowing and non-windowing, we chose only windowing as it has been proven by most recent papers and estimations to be very efficient.

3.1.5 Oblivious Carry Runways

Runways work by splitting the register in half (or however many times necessary) and working on each piece independently in parallel. While similar to coset representation introduces a degree of approximation (about a 2^{-m} deviation from the original addition for each register where m is the size of the runway). As m increases, the deviation decreases, creating a tradeoff between the two. Runways allow us to greatly reduce the depth of the circuit with little overhead (in terms of gates) compared to using special adders like the carry-lookahead adder or ripple-carry adders.

For our estimation, m will be the same as the c_{dev} initially used for the coset representation. We also use a padding between each runway, which is defined by c_{pad}.

3.1.6 Adders

As previously mentioned in the Literature Review, there are two primary types of adders CLA and RCA, but for our work, we choose to use only RCA due to its superiority in terms of T gate count as well as other measures. We compare the cost using both Gidney's modification of Cuccaro's adder [19] as well as Fedoriaka's adder to help us get a better sense of which optimizations are most influential on the cost.

We also use the logical-AND gate (computation and uncomputation) decomposition from [19] for both adders. This construction was originally introduced for creating Toffoli gates, a special kind of three-qubit gate (see Figure 1 of [27]). The Toffoli gate's matrix and circuit representation are provided in Appendix C.

3.2 Cost Estimation

In terms of the actual physical cost implementation, we use a strategy most similar to Gouzien's approach [24] as well as Ekera and Gidney's approach [22], as their goals/purpose of research are similar to that of this paper. Let's now discuss our physical cost estimation, which was generated by "estimate.py".

We will only discuss the estimation for factoring RSA integers, but the same process is applied to Shor's and Ekera and Hastad's variations for the discrete log problem (DLP), Schnorr DLP, and short DLP (as the only difference is the n_e required), which are all used in cryptographic schemes.

Non-Tunable Parameters		Tunable Parameters	
n	size of the integer (bits)	w_e	size of exponent window
n_e	size of the exponent (bits)	w_m	size of multiplication window
		c_{sep}	padding for runways
		c_{dev}	deviation for coset representation and runway
		d	code distance

At this point, we've talked about each of the parameters besides code distance. Code distance pertains to QEC, and the surface code essentially tells us "how many errors" our quantum code be able to correct before it is "overloaded".

Gate counts are essential to determine the theoretical performance of the algorithm since costly gates like Toffolis dominate the cost of the algorithm. Therefore, creating an abstract measure of the gate counts using the optimal parameters and knowledge of the algorithm gives us a way to compare our implementation to that of previous papers.

Since we use windowed arithmetic for both exponentiation and multiplication, we can eventually break down the whole Modular Exponentiation (ME) into lookup additions (refer to Section 3.1.4 for definition) of which we calculate the count of and multiply with its gate counts to find the total gate count of ME. The equations we show below have been derived in Appendix B.

$$\text{LookupAddtionCount}(n, n_e) = \frac{2n_e n}{w_e w_m} \times \frac{c_{\text{sep}} + 1}{c_{\text{serp}}} + O(\frac{n \log n}{w_e w_m}) \tag{1}$$

Most notably, the use of Fedorkia's adder [16] is constant (1 no matter the number of bits), which is much better than the Cuccaro adder ($2n$). The tradeoff is in the number of classically controlled X gates.

$$\text{ToffoliCount}(n, n_e) \approx \text{LookupAddtionCount}(n, n_e) \times (1 + c_{\text{dev}} \frac{n}{c_{\text{sep}}} + 2^{w_e + c_{\text{dev}}}) \tag{2}$$

We looked at a couple of other gate counts, and their formulas will be derived in n the next section, the parameters we deemed the most optimal from our simulation (and ranking by megaqubitdays) will be substituted into the equations for gate count to give us another measure of difference.

Alongside our gate counts we look at megaqubitdays which is equivalent to $\frac{n_l}{1000} \times \frac{n_h}{24}$ where n_l represents number of logical qubits and n_h the number of hours required to factor which are both generated by the ancillary file "estimates.py".

3.3 Comparisons

We draw comparisons between two previous works (Craig Gidney and Martin Ekera's, as well as Elizen Gouzien's and Nicolas Saoungard). We take the data they generated for their estimations and compare it to ours. We also compared gate counts and overall approaches. It is noted that for some instances, we had to use their code provided in the ancillaries section to recreate their data (using their parameters). Why these two among other estimations previously mentioned in Table 1, because if the estimation already had a high cost for a smaller size integers like 2048-bits then we assumed it would be higher for larger integers. For gate counts, we relied on the equations given or created an equation based on the description of their construction. Also, since Gouizen's work didn't choose

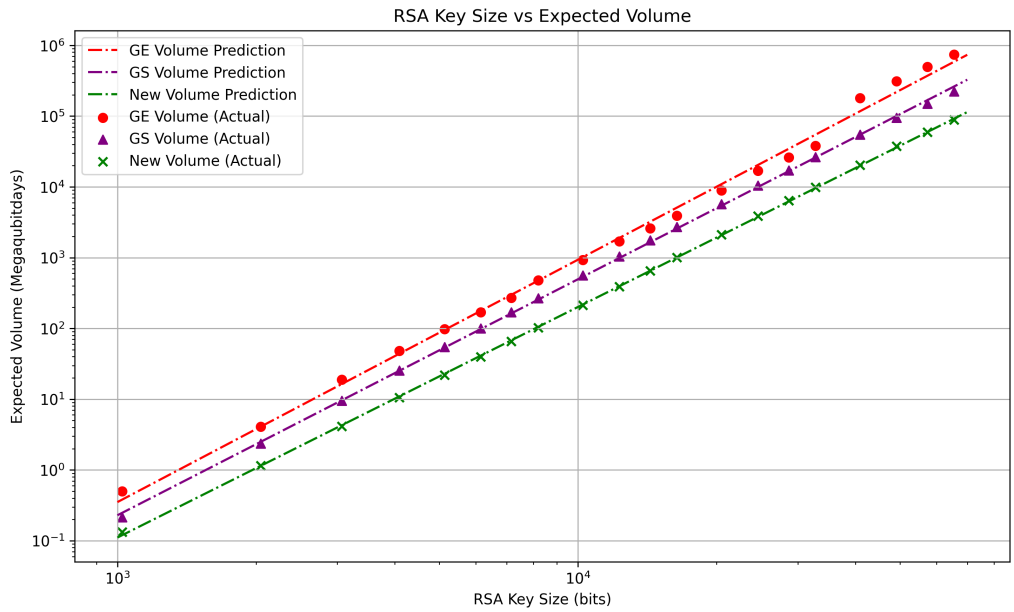


Figure 2: Log log graph with problem size in bits vs spacetime volume for our estimate as well as two others. Equations for each estimate from top to bottom: $Y = 1.857\text{e}{-11}(X)^{3.427}$, $Y = 2.252\text{e}{-11}(X)^{3.336}$, and $Y = 1.856\text{e}{-11}(X)^{3.258}$. Generated by ancillary files "tests.py" and "estimate.py".

parameter values for the gate equations, we chose the values he found for factoring a 2048-bit integer found in his Table 1.

4 Results

4.1 Spacetime Volume Comparison

To better understand the numerical differences between our cost estimates, we used Ordinary Least Squares (OLS) linear regression, which is the execution aspect of linear regression. It aims to reduce the "loss" or error of predictions. The equation below is used to model the predilections

$$Y = \alpha X^{\beta}$$

where Y is the spacetime volume in megaqubitdays and X is the number of bits to be factored, we fit OLS regressions to our log-log plots. The slope β captures how quickly the spacetime volume grows with bit size, while the intercept α represents the predicted megaqubitdays for factoring a one-bit number, which is not meaningful for our purposes. All prediction models fit very well, with R^2 and adjusted R^2 [7] values ≥ 0.997.

Referring to Figure 2, we observe a significant reduction in β from GE's estimate (3.427) and GS's (3.336) to ours (3.258). Although these reductions may appear small, since β appears in the exponent, even minor decreases translate into substantial improvements at large bit sizes. For instance, doubling the problem size leads to a 9.6× increase in our estimation, compared to 10.1× and 10.7× in the others.

[7] Same as R^2 but penalizes for overfitting by accounting for the number of observables and predictors.

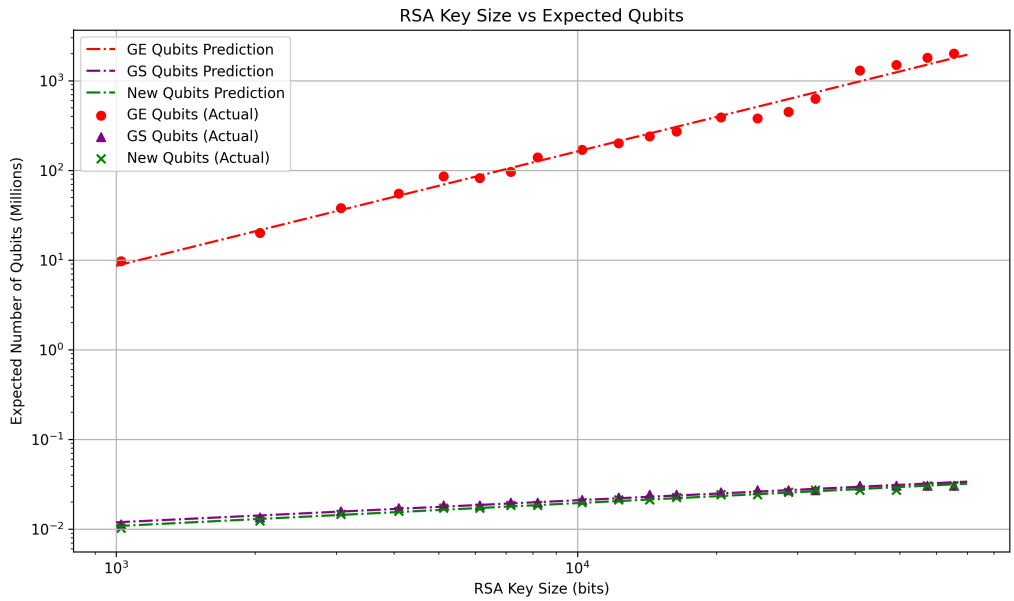


Figure 3: Log log graph with problem size in bits vs estimated qubits in millions for our estimate as well as two others. Equations for each estimate from top to bottom: $Y = 0.0013(X)^{1.276}$, $Y = 0.0022(X)^{0.246}$, and $Y = 0.0018(X)^{0.256}$. Generated by ancillary files "tests.py" and "estimate.py".

Since our research focuses specifically on factoring RSA integers of at least 2048 bits, the smaller β value has an increasingly pronounced impact. At such high bit sizes, the slower growth rate in spacetime volume becomes a key advantage, outweighing small differences in intercepts and leading to better asymptotic performance.

Our R^2 and adjusted R^2 being very close to 1 shows that our equation is extremely accurate yet doesn't overfit the data. As a result, we can safely conclude that our estimation reduces the spacetime volume by a large factor compared to previous estimates, as our hypothesis stated.

4.2 Alternate Comparisons

While we did compare the spacetime volume of each of our estimations and GE's and GS's, we noticed that there wasn't much change in the qubits from ours compared to previous estimates. Accordingly, we assumed that this was due to a large decrease in time, and so this section is dedicated to discovering which factor was more influential in changing our spacetime volume to determine whether or not our hypothesis is true.

Looking at Figure 3, we see a massive difference between GE's and the other two estimates caused by the difference in β values (1.03 and 1.02). Contrary to our hypothesis, we didn't see a drastic difference compared to GS's estimate. Using the example of doubling the problem size, the difference of the scaling factors is 0.01, reinforcing that our estimate didn't improve the qubit count by the "large" amount we had predicted it to in our hypothesis.

Figure 4 shows the complete opposite story, GE's estimation requires the least amount of days, while the other two estimations have very similar high estimates. From the graph, we observe that as the problem size (in bits) increases, the estimated number of days required for computation grows more significantly for the second and third estimations,

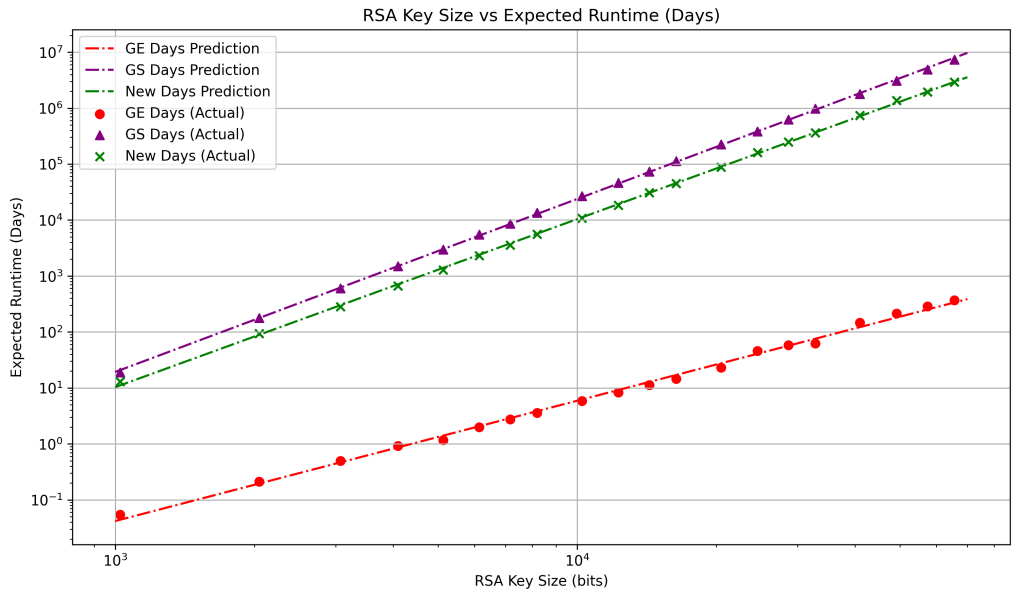


Figure 4: Log log graph with problem size in bits vs estimated days for our estimate as well as two others. Equations for each estimate from top to bottom: $Y = 1.040\text{e}{-8}(X)^{3.090}$, $Y = 1.015\text{e}{-8}(X)^{3.002}$, and $Y = 1.492\text{e}{-8}(X)^{2.149}$. Generated by ancillary files "tests.py" and "estimate.py".

which is reflected in the values of β (3.090 and 3.002) compared to GE's estimate (2.149). This suggests that the estimation from GE, with its lower exponent, is much more efficient in terms of scaling with problem size.

Ours and GS's estimates tended to be together because of our method of error correction. Most notably, we use the same 3D color gauge surface code that doesn't require distillation. GE, on the other hand, uses a 2D surface code with T state distillation. Reduction of spacetime volume compared to the other estimates we now have is achieved because we made minor improvements on both the qubit count and days to GS's count.

4.3 Gate Count Comparison

Next, we look at several gate counts that are a good measure of how well the circuit is, as they dominate its cost. Previous analysis (reference here) looked at CNOTs, AND computation and un-computation, Toffoli gates, and 1 or 2 qubit gates. We used the parameters (shown in Table 2) that were found within the experiment. Each parameter had a range they could vary within (see Appendix D for code). We chose the most frequent number (mode), hence $w_e = 3, w_m = 2, c_{\text{sep}} = 512$, and $c_{\text{pad}} = 36$ and 39. Our reasoning for choosing two numbers for c_{pad} was that the values varied with the size of n so we chose a mid-range value. Finally,Table 2 only goes up to $n = 16384$ because that's one of the last most reasonable numbers. Larger numbers are simply used within graphs to give us better visuals, and because more data points mean a more meaningful prediction.

		Parameters				
n	n_e	w_e	w_m	c_{sep}	c_{dev}	d
1024	1493	3	2	512	28	43
2048	3029	3	2	512	31	47
3072	4565	3	2	512	33	49
4096	6101	3	2	512	34	53
5129	7652	3	2	512	35	53
6144	9173	3	2	512	36	55
7168	10709	3	2	512	36	57
8192	12245	3	2	512	37	57
10240	15317	3	2	512	38	59
12288	18389	3	2	512	39	61
14336	21461	3	2	512	39	61
16384	24533	3	2	512	39	63

Table 2: The table contains values for the parameters used in equations (except d). The values were generated by "estimate.py" and turned into a table by "latexgen.py". A version with the estimations for megaqubitdays, hours, and qubits can be found in the code repository linked in Appendix D as well as tables for larger key sizes.

$$\begin{aligned}
\text{ToffoliCount}(n) &\approx 0.45n^2 + 0.0495n^3 \\
\text{ToffoliCount}_{\text{GE}}(n) &\approx 0.3n^3 + 0.0005n^3 \log_2 n \\
\text{ToffoliCount}_{\text{GS}}(n) &\approx 592n + 39.467n^2 + 0.657n^3
\end{aligned}$$

As n approaches larger numbers (or ∞), the smaller terms become insignificant, so we focus on the largest term n^3 and its coefficients. Since $0.0495 < 0.3 < 0.657$, we can safely say that as n gets larger, our Toffoli count will always be less than the previous constructions. And as the key sizes get larger, so do the differences, varying by a whole factor of 10 for most common key sizes, which supports my initial hypothesis that there would be a substantial decrease.

4.4 Error Rates

A possible dilemma that arose was the choice of physical gate error rate, as previous estimations use (10^{-3}) while newer quantum computers have reached lower rates. For example, researchers at the Delft University of Technology achieved an error rate of (10^{-5}) with spin qubits [6], which is why we also accounted for this possibility when making estimations (see Figure 3). We also looked at scaling based on common error rates ranging from 10^{-2} to 10^{-7}.

Every change that we make to the error rate only affects the final estimation, as it changes how much error correction we need to do with our surface code. As you can see within the graph, the most noticeable jumps are from 10^{-3} to 10^{-4} and 10^{-4} to 10^{-5}. Interestingly enough, the difference between 10^{-5}, 10^{-6}, and 10^{-7} is relatively low, meaning the surface code is approaching a performance bound; anything below it would be theoretical. Another possible explanation is that lowering the error rate allowed the quantum computer to operate below the fault tolerance threshold. This means it can

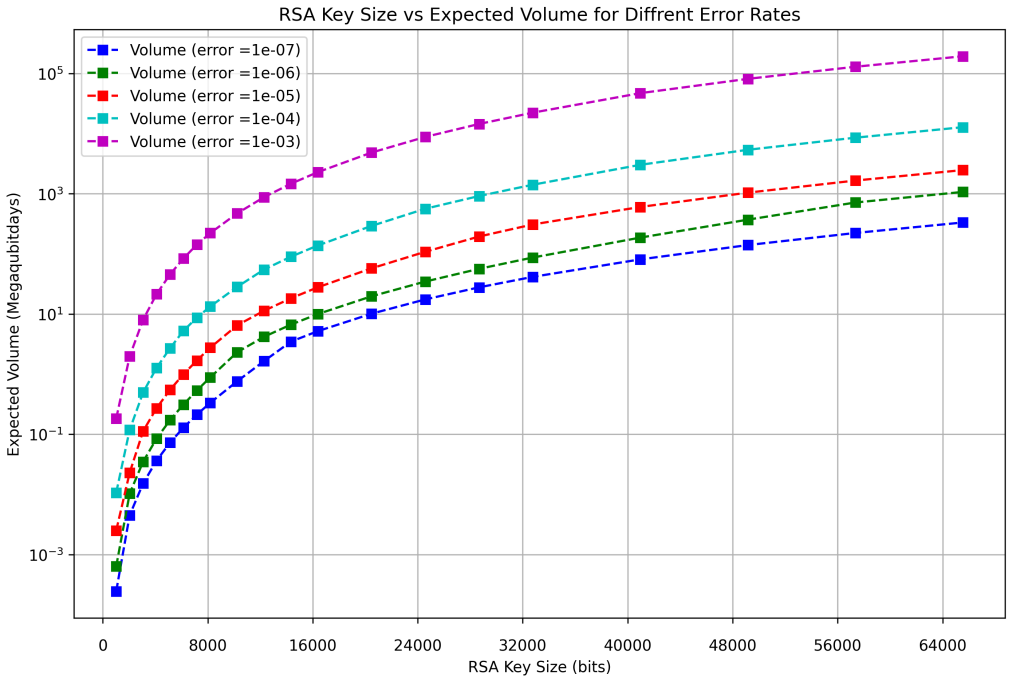


Figure 5: Graph of the hours required to factor various sizes of RSA integers and each line corresponding to a different error rate. The y-axis uses a logarithmic scale, and Ekera-Hastad's variation of Shor's algorithm is used. Generated by ancillary files "extrapolate.py" and "estimate.py". Please note that the lines between the data points are not predilections.

perform error correction that suppresses logical errors to extremely low levels, reducing the number of physical qubits required

The physical assumptions researchers make (mentioned at the beginning of the methodology) are usually a limitation, but we attempt to combat this issue by using several error rates. While looking at error rates may not directly support the research question, it provides a baseline comparison for future researchers.

4.5 Limitations

Our largest limitation within our results is not accounting for deviation and success rates. As mentioned before, runways and coset representation are both approximations of the original circuit, which is good for speeding up the process, but it increases the chance of error. Papers like GE account for this with a formula for total deviation, yet because of time, we weren't able to address the issue. In addition to the approximation deviation, if a substantial amount of error occurs, there is a chance we may have to retry the entire operation (often called retry risk ϵ), which again we didn't calculate for due to time limitations.

Another limitation is that the field is constantly evolving, which is why we chose not to pick optimizations that have just been developed or those that have been constantly used, as they wouldn't help. However, newer techniques for reducing the number of qubits required for logical-CNOTs and other operations, like those by Gideny [23] and other researchers, would be notable additions to future estimations.

4.6 Review of Contributions

While some of the results pertaining to qubit count and days to factor may have been unexpected, other results, like gate count and spacetime volume, were well predicted by our hypothesis. Although we did see a decrease in spacetime volume and gate count, which confirmed our hypothesis, our claim about a large reduction of qubit count and depth wasn't completely true, which is why our results only confirm part of our hypothesis. By addressing RSA keys of size ≥ 2048 bits, we fulfilled the gap presented in our Literature review. Further analysis of gate error rates broadened our results, making them applicable even as quantum computers continue to improve.

Most notably, our results suggest that the feasibility of breaking RSA depends critically on quantum error correction (QEC) and the development of effective surface codes. A key difference between GE's work and ours (as well as GS's) is that both of the latter use a different type of surface code, highlighting that QEC is the central challenge — at least in our current phase of quantum computing. Furthermore, even as surface codes improve, there is no linear scaling with circuit size (as seen in the paper). As a result, even once 2048-bit integers become factorable, there will likely be a significant delay before other commonly used key sizes also become vulnerable.

5 Conclusions and Future Directions

As discussed above, we investigated RSA, applying EH's variation of Shor's algorithm to factor integers. One important area we have not yet emphasized is elliptic curve cryptography (ECC), which supports systems like the Bitcoin blockchain. Newer research has explored adapting methods from Ekera, Regev, and Shor's work to tackle newer cryptographic schemes, such as ECC and SHA-256, or even problems related to integer factorization, such as the discrete log problem [5]. Understanding the viability of quantum attacks on these algorithms is vital, as they could become a weapon for threat actors in the future.

5.1 Cryptographic Problems and Optimizations

Surface codes themselves are always changing, each offering different trade-offs. The primary reason we chose our specific surface code over newer alternatives is that it supports a basis of gates sufficient for universal quantum computation without requiring magic state distillation, which is a significant cost-saving advantage. However, future work should explore other surface codes and techniques, especially incorporating the Topological Quantum Error Correction (TQEC) software. TQEC provides a high-level abstraction for designing and compiling large-scale quantum computations using lattice surgery, helping automate circuit construction and simulate scaled, noisy computations efficiently.

One important optimization we did not take complete advantage of is Fedorkiá's adder, which significantly reduces the T-gate count. Optimizing the number of T gates is essential because, in most surface codes, T gates require costly distillation processes. Surface codes without natural support for T gates still dominate the field, making low T-count algorithms especially valuable.

In addition, alternative types of qubits, such as cat qubits and other bosonic qubits, could dramatically reduce the number of physical qubits required. Unlike transmon qubits (what we used), bosonic qubits can encode information in more than two levels, opening the door to constructions with fewer qubits. However, research into bosonic surface codes remains limited [25, 29, 45].

5.2 Implications for Cryptographic Security

Cost estimation offers a viable way to track progress towards breaking RSA and other cryptographic schemes. By comparing theoretical resource estimates with the capabilities of current quantum computers, we can assess how much time remains before large-scale quantum attacks become feasible.

Given that transitioning the entire internet to quantum-resistant cryptography could take many years, finding less disruptive interim strategies is essential. One approach is to increase RSA key sizes rather than immediately switching to entirely new schemes. Our results show that tripling or quadrupling the standard 2048-bit RSA key size causes the resource requirements, in both qubits and runtime, to increase almost exponentially. Even with parallelization techniques such as runways to save circuit depth, the scaling behavior remains largely unchanged.

Ultimately, the timeline for breaking RSA will depend on the speed at which quantum hardware advances. While we do not attempt to predict when RSA will break, future researchers can estimate the time when it will be broken by continuously comparing cost estimates to correct quantum technology.

5.3 Challenges to Overcome

It is important to understand that we are still far from breaking RSA. Current quantum error correction methods are not yet capable enough: physical errors occur too frequently, and modern surface codes cannot correct them fast enough to enable large-scale computations.

One ongoing problem for quantum computation is addressing environmental noise, e.g., disturbances from cosmic rays. High-energy radiation causes large, correlated error bursts, which are far more damaging than the small, isolated errors that current QEC schemes are designed to correct [32]. This problem only worsens as processors scale up in size.

Even if quantum computers achieve the necessary physical qubit counts and error rates, decoherence will remain a fundamental challenge. As quantum circuits grow deeper, maintaining coherence becomes exponentially harder.

In short, major advances in quantum error correction and hardware design are still required before large-scale cryptographic attacks like breaking RSA become realistic.

References

[1] M. AbuGhanem. Ibm quantum computers: Evolution, performance, and future directions, 2024.

[2] G. Q. Acharya, Rajeev AI and Collaborators. Quantum error correction below the surface code threshold. *Nature*, Dec 2024.

[3] R. Acharya and G. Q. AI. Suppressing quantum errors by scaling a surface code logical qubit. *Nature*, 614(7949):676–681, Feb 2023.

[4] M. Amico, Z. H. Saleem, and M. Kumph. Experimental study of shor's factoring algorithm using the ibm q experience. *Physical Review A*, 100(1), July 2019.

[5] R. Barbulescu, M. Barcau, and V. Pasol. A comprehensive analysis of regev's quantum algorithm. Cryptology ePrint Archive, Paper 2024/1758, 2024.

[6] H. Bartling, J. Yun, K. Schymik, M. van Riggelen, L. Enthoven, H. van Ommen, M. Babaie, F. Sebastiano, M. Markham, D. Twitchen, and T. Taminiau. Universal high-fidelity quantum gates for spin qubits in diamond. *Phys. Rev. Appl.*, 23:034052, Mar 2025.

[7] S. Beauregard. Circuit for shor's algorithm using 2n+3 qubits, 2003.

[8] J. Bos and M. Stam. *Computational Cryptography*, page 41–77. Cambridge University Press, Oct. 2021.

[9] J. P. Buhler, H. W. Lenstra, and C. Pomerance. Factoring integers with the number field sieve. In A. K. Lenstra and H. W. Lenstra, editors, *The development of the number field sieve*, pages 50–94, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg.

[10] M. Case. A beginner's guide to the general number field sieve. *Oregon State University, ECE575 Data Security and Cryptography Project*, 2003.

[11] S. A. Cuccaro, T. G. Draper, S. A. Kutin, and D. Petrie Moulton. A new quantum ripple-carry addition circuit. *arXiv e-prints*, pages quant–ph/0410184, Oct. 2004.

[12] T. G. Draper. Addition on a quantum computer, 2000.

[13] M. Ekerå. On post-processing in the quantum algorithm for computing short discrete logarithms. *Designs, Codes and Cryptography*, 88(11):2313–2335, Nov 2020.

[14] M. Ekerå and J. Håstad. Quantum algorithms for computing short discrete logarithms and factoring rsa integers. In T. Lange and T. Takagi, editors, *Post-Quantum Cryptography*, pages 347–363, Cham, 2017. Springer International Publishing.

[15] M. Ekerå and J. Gärtner. A high-level comparison of state-of-the-art quantum algorithms for breaking asymmetric cryptography, 2024.

[16] D. Fedoriaka. New circuit for quantum adder by constant, 2025.

[17] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Phys. Rev. A*, 86:032324, Sep 2012.

[18] V. Gheorghiu and M. Mosca. Benchmarking the quantum cryptanalysis of symmetric, public-key and hash-based cryptographic schemes, 2019.

[19] C. Gidney. Halving the cost of quantum addition. *Quantum*, 2:74, June 2018.

[20] C. Gidney. Approximate encoded permutations and piecewise quantum adders, 2019.

[21] C. Gidney. Windowed quantum arithmetic, 2019.

[22] C. Gidney and M. Ekerå. How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021.

[23] C. Gidney, M. Newman, P. Brooks, and C. Jones. Yoked surface codes, 2023.

[24] E. Gouzien and N. Sangouard. Factoring 2048-bit rsa integers in 177 days with 13 436 qubits and a multimode memory. *Phys. Rev. Lett.*, 127:140503, Sep 2021.

[25] Gouzien, D. Ruiz, F.-M. Le Régent, J. Guillaud, and N. Sangouard. Performance analysis of a repetition cat code architecture: Computing 256-bit elliptic curve logarithm in 9 hours with 126 133 cat qubits. *Physical Review Letters*, 131(4), July 2023.

[26] G. H. Hardy and E. M. Wright. *An introduction to the theory of numbers*. Oxford University Press, 1979.

[27] C. Jones. Low-overhead constructions for the fault-tolerant toffoli gate. *Phys. Rev. A*, 87:022328, Feb 2013.

[28] N. C. Jones, R. Van Meter, A. G. Fowler, P. L. McMahon, J. Kim, T. D. Ladd, and Y. Yamamoto. Layered architecture for quantum computing. *Phys. Rev. X*, 2:031007, Jul 2012.

[29] J. Lee, N. Kang, S.-H. Lee, H. Jeong, L. Jiang, and S.-W. Lee. Fault-tolerant quantum computation by hybrid qubits with bosonic cat code and single photons. *PRX Quantum*, 5(3), Aug. 2024.

[30] E. Lucero, R. Barends, Y. Chen, J. Kelly, M. Mariantoni, A. Megrant, P. O'Malley, D. Sank, A. Vainsencher, J. Wenner, T. White, Y. Yin, A. N. Cleland, and J. M. Martinis. Computing prime factors with a josephson phase qubit quantum processor. *Nature Physics*, 8(10):719–723, Oct 2012.

[31] A. Luongo, A. M. Miti, V. Narasimhachar, and A. Sireesh. Measurement-based uncomputation of quantum circuits for modular arithmetic, 2024.

[32] M. McEwen, L. Faoro, K. Arya, A. Dunsworth, T. Huang, S. Kim, B. Burkett, A. Fowler, F. Arute, J. Bardin, A. Bengtsson, A. Bilmes, B. B. Buckley, N. Bushnell, J. Chen, R. Collins, S. Demura, A. R. Derk, C. Erickson, M. Giustina, S. Harrington, S. Hong, E. Jeffrey, J. Kelly, P. V. Klimov, F. Kostritsa, P. Laptev, A. Locharla, X. Mi, K. C. Miao, S. Montazeri, J. Mutus, O. Naaman, M. Neeley, C. Neill, A. Opremcak, C. Quintana, N. Redd, P. Roushan, D. Sank, K. Satzinger, V. Shvarts, T. White, J. Yao, P. Yeh, J. Yoo, Y. Chen, V. Smelyanskiy, J. Martinis, H. Neven, A. Megrant, L. Ioffe, and R. Barends. Resolving catastrophic error bursts from cosmic rays in large arrays of superconducting qubits. *Nature Physics*, 2021.

[33] T. Monz, D. Nigg, E. A. Martinez, M. F. Brandl, P. Schindler, R. Rines, S. X. Wang, I. L. Chuang, and R. Blatt. Realization of a scalable shor algorithm. *Science*, 351(6277):1068–1070, 2016.

[34] J. O'Gorman and E. T. Campbell. Quantum computation with realistic magic-state factories. *Phys. Rev. A*, 95:032338, Mar 2017.

[35] A. Politi, J. C. F. Matthews, and J. L. O'Brien. Shor's quantum factoring algorithm on a photonic chip. *Science*, 325(5945):1221–1221, 2009.

[36] C. Pomerance and P. Erdös. A tale of two sieves, 1998.

[37] S. Ragavan and V. Vaikuntanathan. Space-efficient and noise-robust quantum factoring, 2024.

[38] O. Regev. An efficient quantum factoring algorithm, 2024.

[39] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

[40] J. Roffe. Quantum error correction: an introductory guide. *Contemporary Physics*, 60(3):226–245, 2019.

[41] P. W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, Oct. 1997.

[42] R. VAN METER, T. D. LADD, A. G. FOWLER, and Y. YAMAMOTO. Distributed quantum computation architecture using semiconductor nanophotonics. *International Journal of Quantum Information*, 08(01n02):295–323, Feb. 2010.

[43] L. M. K. Vandersypen, M. Steffen, G. Breyta, C. S. Yannoni, M. H. Sherwood, and I. L. Chuang. Experimental realization of shor's quantum factoring algorithm using nuclear magnetic resonance. *Nature*, 414(6866):883–887, Dec 2001.

[44] V. Vedral, A. Barenco, and A. Ekert. Quantum networks for elementary arithmetic operations. *Phys. Rev. A*, 54:147–153, Jul 1996.

[45] Q. Xu, P. Zeng, D. Xu, and L. Jiang. Fault-tolerant operation of bosonic qubits with discrete-variable ancillae, 2023.

[46] C. Zalka. Shor's algorithm with fewer (pure) qubits, 2006.

Appendices

A Algorithm Outlines

A.1 Shor's Algorithm

His paper, published in 1991, detailed the factoring of integers by solving the order-finding problem faster on a Quantum Computer (QC), since it had previously been proven that factoring can be reduced to order-finding.

Given $N \in \{2k+1 : k \in \mathbb{Z}\}$ (N is odd), N can be written as the product of two primes p and q such that $p \neq q$. To solve this problem, we begin by doing the following classically:

1. Choose a random integer a such that $1 < a < N$.

2. If $\gcd(a, N) > 1$, then $\{p, q\} = \{\gcd(a, N), \frac{N}{\gcd(a,N)}\}$ and the factors are found. Otherwise, $\gcd(a, N)$ must equal 1 and the factors can be found by finding the order using a QC.

   The **order** of an element a in $\mathbb{Z}_n^*$ (denoted $\mathrm{ord}(a)$) is the smallest positive integer r such that $a^r \equiv 1 \pmod{n}$.

3. **Quantum Computation**: The circuit uses two registers, a period, and a computational register. We first apply the Hadamard gate on the period register with n qubits initialized to $|0\rangle$ to create an equal superposition:

$$|p\rangle = \frac{1}{\sqrt{2^n}} \sum_{j=0}^{2^n-1} |j\rangle$$

   Then we apply $f(x) = a^r \bmod N$ to the computational register $|c\rangle$, which was initialized to $|1\rangle$, resulting:

$$|c\rangle = |a^j \bmod N\rangle$$

   The result of both registers is:

$$|p\rangle \otimes |c\rangle \equiv |p\rangle|c\rangle$$

$$\equiv \frac{1}{\sqrt{2^n}} \sum_{j=0}^{2^n-1} |j\rangle|a^j \bmod N\rangle$$

   which can be rewritten to emphasize the periodicity:

$$\frac{1}{\sqrt{r2^n}} \sum_{s=0}^{r-1} \sum_{j=0}^{2^n-1} e^{2\pi isj/r} |j\rangle|u\rangle$$

   where $|u\rangle$ is the eigenstate $\frac{1}{\sqrt{2}} \sum_{k=0}^{r-1} e^{\frac{-2\pi isj}{r}} |a^j \bmod N\rangle$ of result of $QFT^\dagger$ on the computation register. After applying $QFT^\dagger$ we have:

$$\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |s/r\rangle|u_s\rangle$$

and after measuring the first register we get s/r, where we can finally apply the continued fractions algorithm to retrieve r.

4. If r is even and $x^{r/2} \neq -1 \pmod N$ then check if $\gcd(x^{r/2}-1, N)$ or $\gcd(x^{r/2}+1, N)$ is a non-trivial factor, if so return the factor, otherwise try again.

B Extra Proofs/Mathematical Evidence

B.1 Cost of the Quantum Fourier Transform

The cost of the QFT function has a polynomial time complexity ($O(n^2)$), which we attempt to show below. The Quantum Fourier Transform (QFT) function transforms the state as follows

$$|j\rangle \xrightarrow{QFT} \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi ijk/2^n} |k\rangle$$

And can be broken down into a series of rotation and Hadamard gates. We apply one Hadamard gate and $n-1$ controlled rotation gates to the first qubit. which is n gates. On the second we apply one Hadamard and $n-2$, which is $n-1$ gates, and continue. Summing up all the gates for each qubit, we have $n(\frac{n+1}{2})$ gates, which is $O(n^2)$ as we stated.

B.2 Modular Exponentiation to Modular Addition

Let's say we wish to compute $x^k \pmod n$, which is to say the remainder from x^k/n. The binary (base 2) expansion of k would be:

$$k = z_t \cdot 2^{t-1} + z_{t-1} \cdot 2^{t-2} + ... + z_1 \cdot 2^0$$

Applying the binary expansion to $x^k \pmod n$ we have:

$$x^k \pmod n = (x^{z_t \cdot 2^{t-1}} \pmod n)(x^{z_{t-1} \cdot 2^{t-2}} \pmod n) ...(x^{z_1 \cdot 2^0} \pmod n)$$

which is simply t-1 modular multiplications. Implementing one modular multiplication, which computes $zx \pmod n$ can be simplified into taking repeated additions, i.e.

$$zx \pmod n = 2^{t-1} zx_t + 2^{t-2} zx_{t-1} + \ ... + 2^0 zx_1$$

Hence, modular exponentiation can be broken down into modular additions.

B.3 Gate Count Formulas

Using windowing there are n_e multiplications done in groups of w_e where $\frac{n_e}{w_m}$ multiplication per "window". Each multiplication consists of 2 multiplications and additions. Each multiply addition does additions controlled by the n register qubits, $O(\log n)$ coset representation qubits, and $\frac{n}{c_{\text{sep}}}$. Each lookup addition is used for w_m groups of additions.

Since each of the additions in the multiple additions is controlled by the three subjects mentioned above, we have:

$$(n + O(\log n) + \frac{n}{c_{\text{sep}}})$$

Since each addition is in groups of w_m and its done 2 times and we have $\frac{n_e}{w_e}$ multiply adds:

$$\frac{(n + O(\log n) + \frac{n}{c_{\text{sep}}})}{w_m}$$

$$2\frac{n_e}{w_e} \times \frac{(n + O(\log n) + \frac{n}{c_{\text{sep}}})}{w_m}$$

Rewriting:

$$2\frac{n_e}{w_e} \times \frac{(n(1 + \frac{1}{c_{\text{sep}}}) + O(n \log n))}{w_m}$$

$$2\frac{n_e(n(1 + \frac{1}{c_{\text{sep}}}) + O(n \log n))}{w_e w_m}$$

Separating into two fractions,

$$2\frac{n_e(n(1 + \frac{1}{c_{\text{sep}}}))}{w_e w_m} + \frac{O(n \log n)}{w_e w_m}$$

$$2\frac{n_e n(\frac{c_{\text{sep}}+1}{c_{\text{sep}}})}{w_e w_m} + \frac{O(n \log n)}{w_e w_m}$$

And finally,

$$\frac{2n_e n}{w_e w_m} \times \frac{c_{\text{sep}} + 1}{c_{\text{sep}}} + O(\frac{n \log n}{w_e w_m})$$

which we use as the formula for lookup addition count. Toffoli count is simply the cost of one lookup addition times the count.

$$\text{ToffoliCount}(n, n_e) \approx \text{LookupAddtionCount}(n, n_e) \times (1 + c_{\text{dev}} \frac{n}{c_{\text{sep}}} + 2^{c_{\text{dev}} + c_{\text{sep}}})$$

C List of Common Quantum Gates

The key element of quantum gates is that the matrix representing them is unitary. In other words, if U is the matrix representation for a gate it obeys the following:

$$U = U^{-1} \quad \text{or} \quad UU^{\dagger} = I$$

In practicality it means that all gates even logical gates (higher level) must have some inverse gate or uncomputation. Please note that in general n-qubit gate means that the gate acts on n qubits. For the description of 3-qubit gates like a Toffoli see the following page.

1-Qubit Gates

Quantum Gate	Circuit representation	Matrix
Pauli-X (X)	$-\boxed{X}-$	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli-Y (Y)	$-\boxed{Y}-$	$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli-Z (Z)	$-\boxed{Z}-$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Hadamard (H)	$-\boxed{H}-$	$\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
Phase (S, P)	$-\boxed{S}-$	$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$
T	$-\boxed{T}-$	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$

Figure 6: Each common one-qubit gate with its respective name, visual representation, and matrix.

2-Qubit Gates

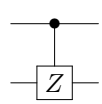
Quantum Gate	Circuit representation	Matrix
Controlled Not (CNOT, CX)		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
Controlled Z (CZ)		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$
SWAP		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Figure 7: Each common two-qubit gate with its respective name, visual representation, and matrix.

Other 2+ qubit gates are less common and often broken down into smaller gates, except the Toffoli and logical-AND gates. To find the matrix for a quantum gate, you can use

projectors and other Pauli matrices to construct them. For example, a Toffoli is simply $(P_0 \otimes \mathbb{I} \otimes \mathbb{I}) + (P_1 \otimes P_0 \otimes \mathbb{I}) + (P_1 \otimes P_2 \otimes X)$ where

$$P_0 = |0\rangle\langle 0| \equiv \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \qquad P_1 = |1\rangle\langle 1| \equiv \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$$

. Figure 8 the deconstruction of the logical-AND gate. We use this version within `estimate.py` as well as the construction of Toffoli in Figure 9.

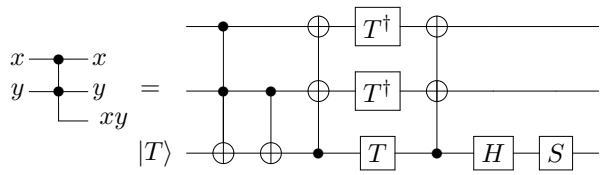


Figure 8: Construction of a logical-AND computation.

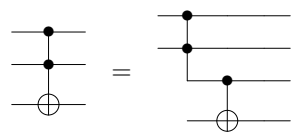


Figure 9: Toffoli Gate construction made up of AND gate on the first two qubits and a CNOT on the result.

D Code

Repository with the source code: link.

The above repository is anonymized and hosted on a public website. Use the sidebar on the left to navigate between the files. The source code contains every graph I generated within `graphs`. The `circuits` folder contains three smaller folders `2025`, which is our construction, `GE2021`, which contains Gideny and Ekera's construction, and `GS21`, which houses Gouzien and Sanougard's work. All three folders contain some form of estimation Python file (e.g. `estimate.py`) and a respective Python file containing classes used within the estimation process (e.g. `structures.py`). Our work and GS's also contain a Python file for error correction codes (`errorcorrection.py`).

Now, most importantly, the OLS results are located in the comments of `tests.py`, which I encourage the reader to read as it contains a plethora of statistical results (`clean.py` is a support Python file for `tests.py`). Each `.csv` file contains an appropriate file description for what it contains. Some of the `.csv`s are regenerated results from the other works (e.g. `gid+ekera(more).csv` and `gou.csv`). Finally, `latexgen.py` was created to generate the LaTeX tables automatically.